

Cosa sono i circuiti sequenziali?

I circuiti sequenziali sono circuiti in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno **stato interno**, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni. In generale, la funzione calcolata dal circuito ad un dato istante dipende dalla sequenza temporale dei valori in input al circuito (storia).

Cosa è un Flip-Flop e come funziona?

Un Flip-Flop è un circuito sequenziale utilizzato come memoria elementare. Possiede uno stato che può essere 0 oppure 1, lo stato dipende dallo stato attuale del Flip-Flop e dal segnale di clock. In genere, il cambio di stato si verifica sul fronte di salita del clock, ossia quando il segnale di clock passa da 0 a 1. Può comunque essere definito nel circuito sequenziale anche sul fronte di discesa.

Dove vengono usati i Flip-Flop e a cosa servono?

I Flip-Flop vengono raggruppati insieme per formare i registri, ovvero memorie in grado di conservare parole con un numero voluto di bit. Lo stesso clock è in ingresso a tutti i flip-flop del registro.

Cosa sono DI0-DI1?

DI0, DI1, ... DIN sono le linee di ingresso dati dei Flip-Flop di un registro, ogni linea corrisponde ad un Flip-Flop.

A cosa serve lo stack e perché è stato inventato da chi ha progettato Risc-V?

Lo stack è una struttura dati dinamica, un'area di memoria in cui è possibile salvare i registri per evitarne la perdita del valore. L'accesso è di tipo LIFO, la cima dello stack è identificata dallo Stack Pointer (SP). In RISC-V si usa la convenzione:

- **grow-down:** lo stack cresce da indirizzi di memoria alti verso indirizzi di memoria bassi
- **last-full:** lo Stack Pointer (SP) contiene l'indirizzo dell'ultima cella di memoria occupata nello stack

Perché ho sia Frame pointer che Stack pointer, e non solo sp?

Il Frame Pointer è utilizzato per identificare la base del frame di una procedura, mentre lo Stack Pointer rappresenta la cima.

Spiegare il funzionamento ed il ruolo dei vari tipi di registri?

RISC-V possiede 32 registri per gli interi, 32 registri per i numeri in virgola mobile, un Program Counter (PC) e 4096 Control Status Register. I ruoli dei 32 registri per gli interi sono così suddivisi:

Registro	Nome	Utilizzo
x0	zero	Costante zero
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Puntatore a thread
x8	s0 / fp	Frame pointer (il contenuto va preservato se utilizzato dalla procedura chiamata)
x10-x11	a0-a1	Passaggio di parametri nelle procedure e valori di ritorno
x12-x17	a2-a7	Passaggio di parametri nelle procedure
x5-x7 x28-x31	t0-t2 t3-t6	Registri temporanei, non salvati in caso di chiamata
x9 x18-x27	s1 s2-s11	Registri da salvare: il contenuto va preservato se utilizzati dalla procedura chiamata

La ALU può leggere e scrivere solo dai registri, l'accesso alla memoria è più lento rispetto a quello dei registri, il compilatore si occupa di individuare la strategia più efficiente per le operazioni di caricamento e salvataggio dei dati tra registri e memoria, variabili utilizzate più di frequente devono rimanere il più possibile salvate nei registri.

Spiegare gerarchia della memoria

La gerarchia della memoria è ordinata secondo la capacità, i costi di memorizzazione e i tempi di accesso all'informazione. È strutturata come una piramide: più in alto troviamo le memorie più piccole e più costose, ma il cui tempo di accesso all'informazione è minore, più in basso troviamo le memorie più capienti e meno costose, ma il cui tempo di accesso all'informazione è maggiore.

- Registri
- Cache
- RAM
- HDD

Come decidiamo cosa mettere nella cache: località temporale e spaziale?

L'uso della memoria di cache è basato sul **principio di località** del codice: i programmi non accedono alla memoria a caso. Le parole di memoria più usate vengono tenute in una cache.

Località temporale: principio secondo cui se si accede ad una determinata locazione di memoria, è molto probabile che vi si acceda di nuovo dopo poco tempo.

Località spaziale: principio secondo cui se si accede ad una determinata locazione di memoria, è molto probabile che si acceda alle locazioni vicine ad essa dopo poco tempo.

Input/Output spiegare i vari modi: busy waiting, interrupt, DMA

La CPU comunica con i device di I/O tramite i **controllori**, che hanno il compito di trasformare i comandi della CPU in segnali elettrici per le periferiche ed i segnali delle periferiche in dati per la CPU.

Ogni controllore ha al suo interno dei **registri** identificati da un indirizzo, che può:

- Appartenere allo stesso insieme di indirizzi di memoria (memory mapped I/O)
- Essere un indirizzo dedicato (isolated I/O)

Registri nei controllori:

- **Dati** (ingresso e uscita): esempio, per una stampante i dati da stampare
- **Comandi** (dalla CPU alla periferica): esempio, i comandi di stampa
- **Informazioni sullo stato della periferica** (dalla periferica alla CPU): esempio, informazione sullo stato della stampante o sull'avanzamento dei lavori

Busy waiting: La CPU controlla lo stato di avanzamento dell'I/O e lo stato della periferica **ispezionando in un ciclo il bit del registro** di stato del controllore READY fino a che questi non segnala di essere pronto per un nuovo comando di I/O. **Svantaggio:** la CPU rimane in ciclo attendendo che il dispositivo sia pronto, da qui il nome busy waiting.

Polling: verifica ciclica (periodica) dei dispositivi di I/O tramite i bit di controllo.

Interrupt: permette di liberarsi dal busy waiting, il dispositivo quando ha finito **genera un segnale** che avverte la CPU di aver completato il proprio lavoro (**interrupt**). La CPU può abilitare l'interrupt mettendo a 1 un opportuno bit. Gli interrupt sono **asincroni** rispetto al programma e devono essere gestiti in modo **trasparente**, ovvero lo stato dell'esecuzione dopo la gestione dell'interrupt deve tornare esattamente come era prima dell'interrupt stesso.

DMA (Direct Memory Access): permette ai dispositivi di I/O di accedere direttamente alla memoria interna per scambiare dati senza coinvolgere la CPU per ogni byte trasferito tramite l'usuale meccanismo dell'interrupt e la successiva richiesta dell'operazione desiderata, ma generando un singolo interrupt per blocco trasferito. Il DMA è impostato direttamente dal software o dal sistema operativo inizializzando opportuni registri, la CPU e il DMA si contendono l'uso del bus.

Differenza tra circuiti sequenziali e combinatori?

Circuiti combinatori: sono in grado di calcolare funzioni che dipendono solo dai dati in input

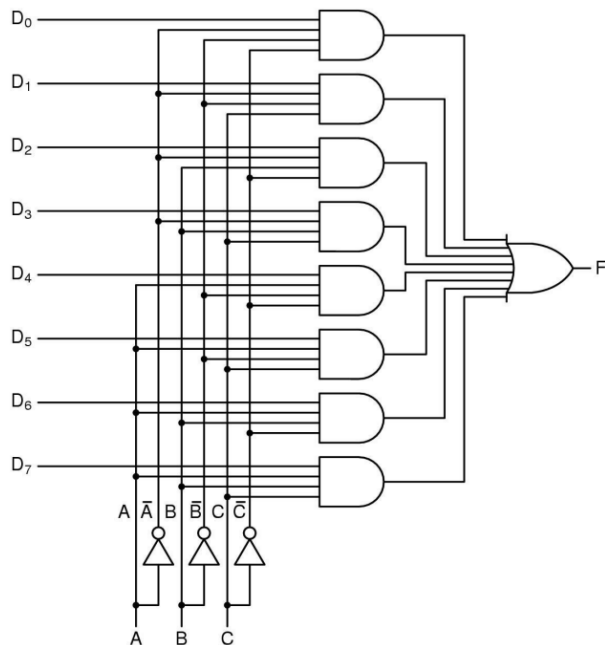
Circuiti sequenziali: sono in grado di calcolare funzioni che dipendono, oltre dai dati in input, anche da uno **stato interno**, ovvero dipendono anche da informazioni memorizzate in elementi di memoria interni.

Disegnare e spiegare il multiplexer

Il multiplexer è un circuito combinatorio che, in generale, possiede 2^n ingressi, 1 uscita e n ingressi di controllo.

Le linee di controllo determinano quale dei 2^n ingressi deve essere selezionato per essere inviato all'uscita.

Esempio con 8 ingressi ($D_0, \dots, D_7$), un'uscita (F) e tre ingressi di controllo (A, B e C):



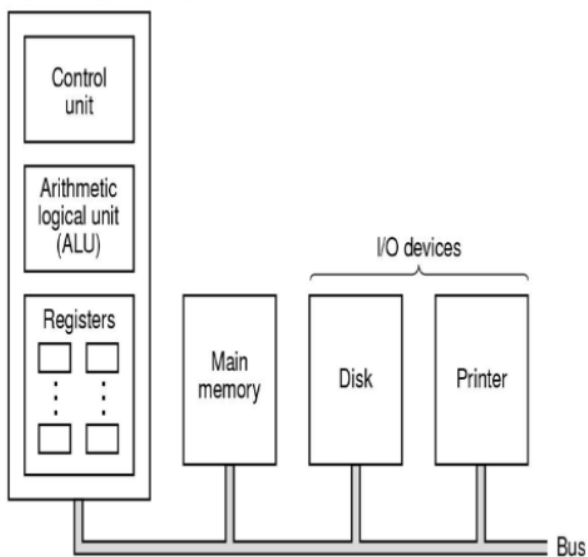
Il multiplexer è sequenziale o combinatorio?

Il multiplexer è un circuito combinatorio.

Spiegare la macchina di Von Neumann

La macchina di Von Neumann si basa su 4 componenti fondamentali: CPU, RAM, Interfaccia di I/O, Bus di sistema.

Central processing unit (CPU)



La CPU si compone di diverse parti distinte: Control Unit, ALU, registri.

I registri, l'ALU e alcuni bus che li collegano compongono il data path.

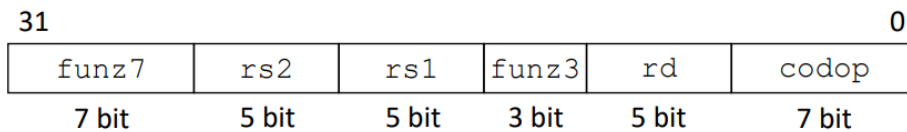
La **main memory** contiene sia istruzioni, sia dati, usando sequenze di bit.

Perché esistono tanti tipi di istruzioni su Risc-V e a cosa servono?

RISC-V definisce diversi formati di istruzioni che consentono di codificare in binario ogni istruzione assembler, ogni istruzione RISC-V richiede esattamente 32 bit. I formati delle istruzioni RISC-V sono R, I, S, SB, U:

Formato di tipo R (registro)

Permette di codificare le istruzioni tra due registri sorgente.



codop: codice operativo dell'istruzione

rd: registro di destinazione

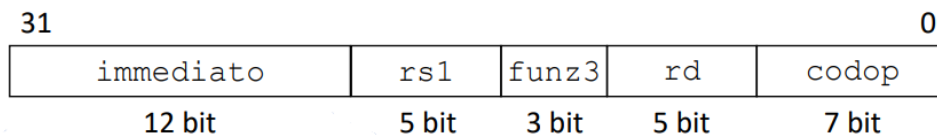
rs1: registro che contiene il primo operando sorgente

rs2: registro che contiene il secondo operando sorgente

funz3, funz7: codici operandi aggiuntivi

Formato di tipo I (immediato)

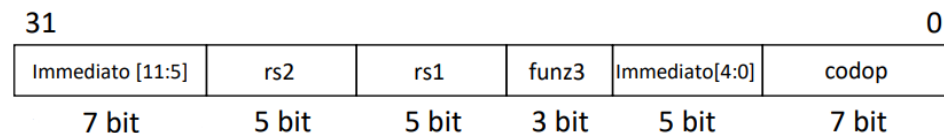
Permette di codificare le istruzioni che richiedono il caricamento dalla memoria o una costante.



Sono presenti 12 bit perché con 5 bit l'intervallo di rappresentazione per costanti e (soprattutto) offset sarebbe stato troppo ridotto. Il campo immediato è rappresentato in complemento a due (valori possibili da -2048 a +2047).

Formato di tipo S

Permette di codificare le istruzioni che richiedono il salvataggio in memoria o una costante.



Il campo immediato viene diviso in due parti per mantenere i due campi rs1 e rs2 nella stessa posizione rispetto alle istruzioni di tipo R. Il campo immediato è rappresentato in complemento a due (valori possibili da -2048 a +2047).

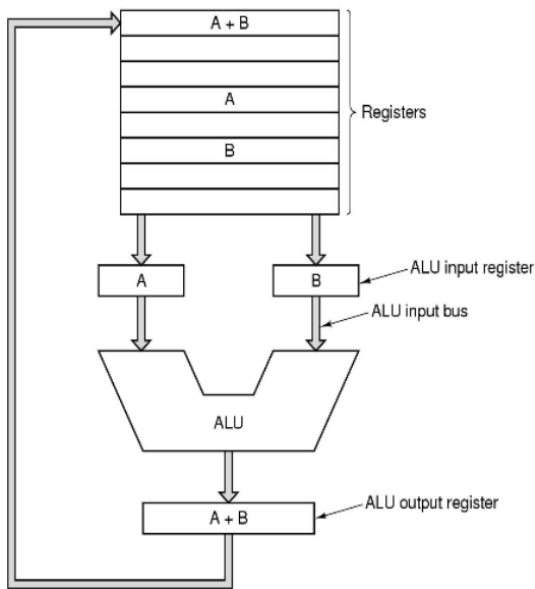
Formato di tipo SB

Permette di codificare le istruzioni di salto condizionato. Il formato può rappresentare indirizzi di salto da -4096 a 4094, in multipli di due.

Formato di tipo U

Permette di codificare le istruzioni di salto condizionato con indirizzo di salto più grandi, un intervallo pari a 2^{32} .

Com'è strutturata la CPU e come funziona?



CPU di Von Neumann

Data path: organizzazione interna di una CPU (registri, ALU, bus interno).

Registro: memoria veloce per dati temporanei.

Istruzioni registro-registro e registro-memoria.

Ciclo del data path: processo di far passare due operandi attraverso l'ALU e memorizzarne il risultato.

Ciclo di fetch-decode-execute

La CPU esegue ogni istruzione del livello 1 (ISA) per mezzo di una serie di passi elementari:

1. Prendi l'istruzione seguente dalla memoria e mettila nel registro delle istruzioni
2. Cambia il program counter per indicare l'istruzione seguente
3. Determina il tipo dell'istruzione appena letta
4. Se l'istruzione usa una parola in memoria, determina dove si trova
5. Metti la parola, se necessario, in un registro della CPU
6. Esegui l'istruzione
7. Torna al punto 1 e inizia a eseguire l'istruzione successiva

Perché evitare di usare la memoria in Risc-V? Quindi come funziona la memoria, dove si trovano memoria e dove registri?

È consigliabile limitare le istruzioni di accesso alla memoria il più possibile, dato che l'accesso alla memoria è più lento rispetto a quello dei registri.

La memoria è un insieme di celle della stessa dimensione, ognuna di esse è identificata da un indirizzo, in RISC-V la memoria è organizzata in parole da 64 bit, in essa possiamo trovare:

- Stack
- Dati allocati dinamicamente (heap)
- Dati statici e variabili globali
- Codice del programma

Lo stack e i dati dinamici (heap) crescono in direzioni differenti, in modo da ottimizzarne la gestione.

I registri si trovano all'interno della CPU.

Cos'è la cache?

La cache è un'area di memoria estremamente veloce ma con una capacità molto bassa. Il suo scopo è quello di velocizzare l'esecuzione dei programmi conservando le parole di memoria più utilizzate da essi. L'uso della memoria cache è basato sul **principio di località** del codice.

Le istruzioni sono salvate in cache?

Sì, è possibile che le istruzioni vengano salvate nella cache.

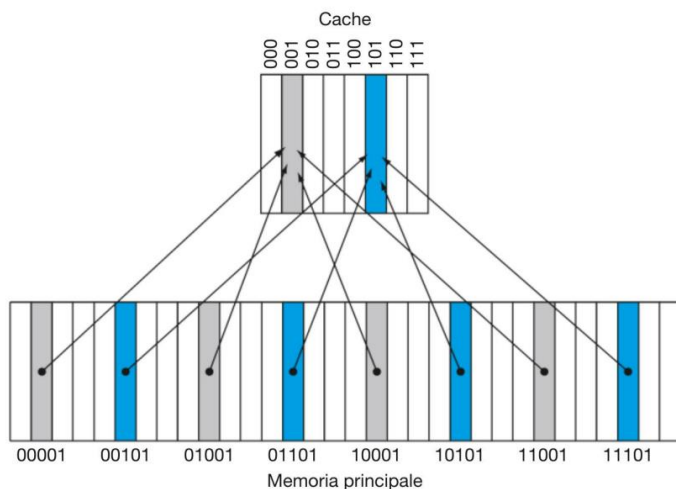
Descrivere i 3 tipi di cache? Come funzionano i principi di località temporale e spaziale nei 3 tipi?

I tre tipi di cache sono:

- Mappatura diretta
- Completamente associativa
- Set-associativa

Mappatura diretta

Una cache in cui ad ogni locazione della memoria principale corrisponde una (e una sola) locazione della cache



1 blocco = 1 parola, in pratica blocchi da k byte

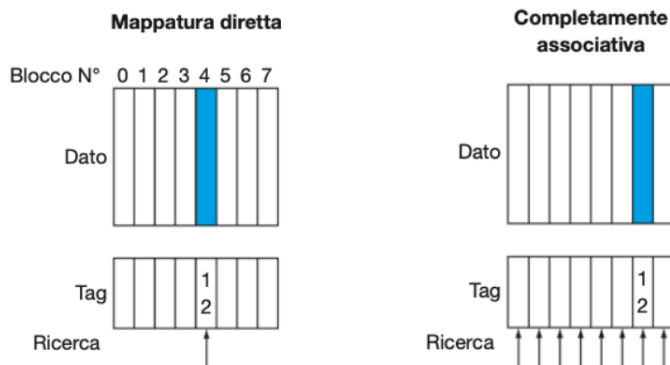
(Indirizzo del blocco) modulo (numero di blocchi della cache)

Come si capisce se in una linea è presente un dato? **Bit di validità**

Come si capisce se nella cache è presente il dato ricercato? **Tag**

Completamente associativa

Un blocco della memoria principale può essere scritto in una qualsiasi locazione della cache

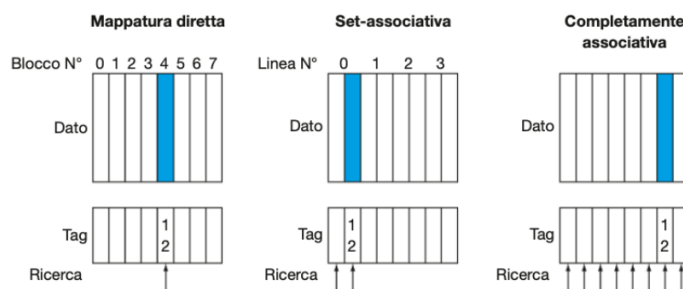


La ricerca va effettuata su tutti gli elementi della cache, svolta in parallelo attraverso un comparatore (aumentando i costi)

Adatta solo per un numero ridotto di blocchi

Set-associativa

Schema intermedio, ogni linea può contenere più blocchi



Individuata la linea, il blocco viene ricercato all'interno degli slot della linea.

(Numero del blocco) modulo (Numero delle linee della cache)

Cosa cambia il numero di linee all'interno della cache?

Il numero di linee all'interno della cache non cambia la dimensione dei dati conservati, bensì semplicemente la quantità di dati conservabili all'interno di essa.

Cos'è un blocco di memoria?

Un blocco di memoria è un insieme di linee (sequenze di byte).

Differenza tra blocco e linea

Un blocco di memoria è un insieme di linee, una linea è una sequenza di byte.

Differenza tra jump e jump and link?

Jump: salta da un'istruzione a quella da eseguire, prosegue dall'istruzione immediatamente successiva a quella a cui si è "saltati".

Jump-and-link: salta da un'istruzione a quella da eseguire, appena termina l'esecuzione dell'istruzione a cui si è saltati, l'istruzione seguente da eseguire è quella subito dopo l'istruzione di jump-and-link.

Come funziona l'unità di controllo? Com'è fatta dentro e quali sono i vari passaggi interni per selezionare la funzione richiesta?

L'unità di controllo è la parte della CPU che contiene i circuiti che effettivamente decodificano le istruzioni.

Ha come input i 7 bit dell'istruzione (**codop**) e ha diversi output (totale 8 bit):

- Due segnali a 1 bit utilizzati per controllare i multiplexer (**ALUSrc** e **MemtoReg**)
- Tre segnali a 1 bit per controllare lettura e scrittura del register file e della memoria dati (**RegWrite**, **MemRead** e **MemWrite**)
- Un segnale a 1 bit utilizzato come segnale di controllo per i salti condizionati (**Branch**)
- Un segnale di controllo a 2 bit per la ALU (**ALUOp**)

Codice operativo (codop): il campo che denota il tipo di operazione e il formato di un'istruzione. A seconda del codop, il campo funz3 e funz7 servono come campi di **estensione del codice operativo**.

AluOp: bit utilizzati come ingressi per selezionare l'operazione che la ALU deve effettuare. AluOp + Funz = Controllo della ALU.

Valori delle linee di controllo

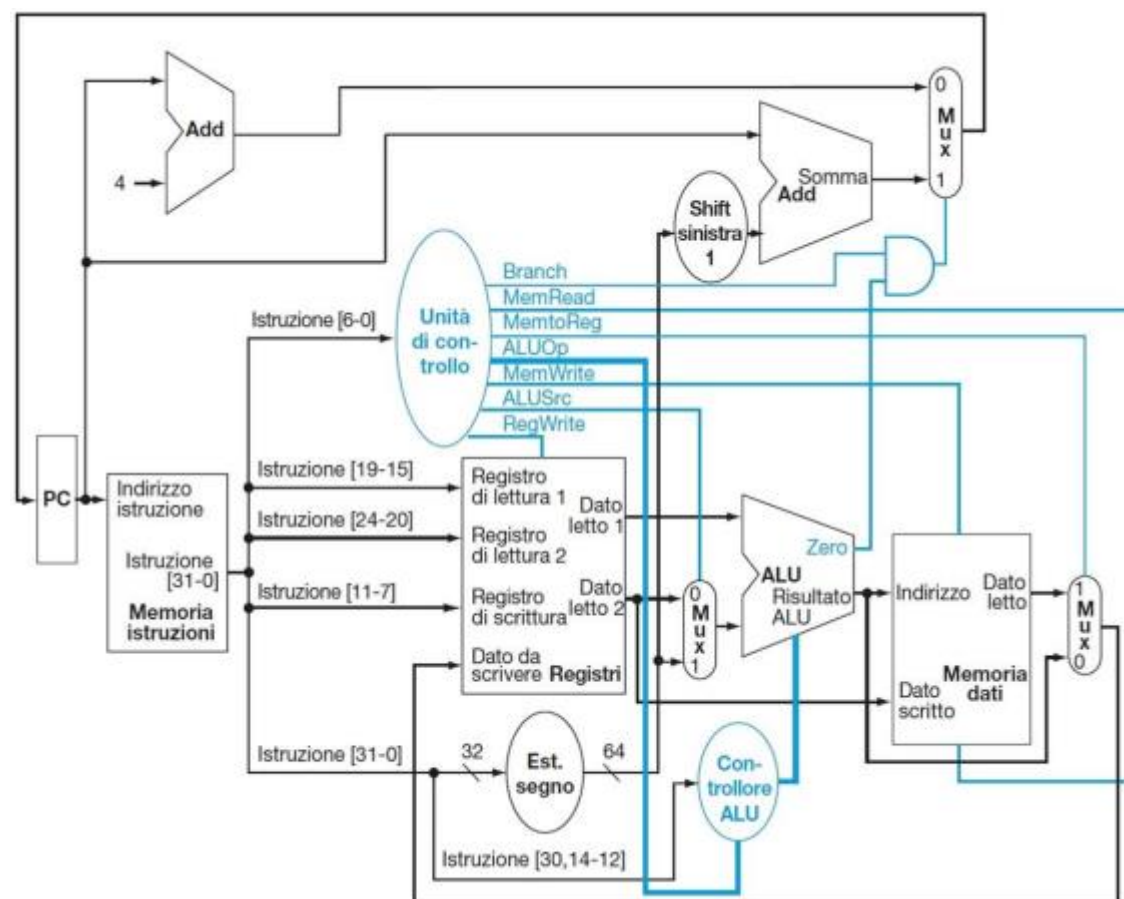
Istruzione	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp1	ALUOp0
Tipo R	0	0	1	0	0	0	1	0
ld	1	1	1	1	0	0	0	0
sd	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

Effetto di ciascuno dei restanti sei segnali di controllo

Non asserito = 0; Asserito = 1

Nome del segnale	Effetto quando non asserito	Effetto quando asserito
RegWrite	Nulla	Il dato viene scritto nel register file nel registro individuato dal numero del registro di scrittura
ALUSrc	Il secondo operando della ALU proviene dalla seconda uscita del register file (Dato letto 2)	Il secondo operando della ALU proviene dall'estensione del segno dei 12 bit del campo immediato dell'istruzione
PCSrc	Nel PC viene scritta l'uscita del sommatore che calcola il valore di $PC + 4$	Nel PC viene scritta l'uscita del sommatore che calcola l'indirizzo di salto
MemRead	Nulla	Il dato della memoria nella posizione puntata dall'indirizzo viene inviato in uscita sulla linea "Dato letto"
MemWrite	Nulla	Il contenuto della memoria nella posizione puntata dall'indirizzo viene sostituito con il dato presente sulla linea "Dato scritto"
MemtoReg	Il dato inviato al register file per la scrittura proviene dalla ALU	Il dato inviato al register file per la scrittura proviene dalla Memoria Dati

Schema dell'unità di controllo completa



Spiegare e disegnare half adder e full adder?

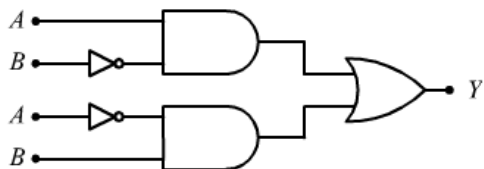
I sommatore binari sono reti combinatorie che ricevono in ingresso n bit degli addendi da sommare e generano in uscita i bit della somma binaria con il relativo riporto.

$$Y = A \oplus B = A\bar{B} + \bar{A}B$$



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

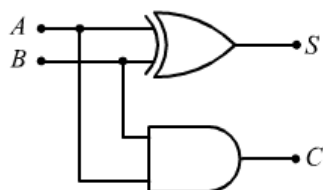
- $0 + 0 = 0$
- $0 + 1 = 1$
- $1 + 0 = 1$
- $1 + 1 = 0$ con riporto 1



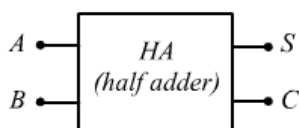
Questo schema non è altro che la tabella di verità della porta XOR (a parte la questione del riporto che deve essere risolta).

Half-Adder

Per gestire anche il riporto questo circuito deve essere modificato in modo da risolvere la seguente tabella della verità:



A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

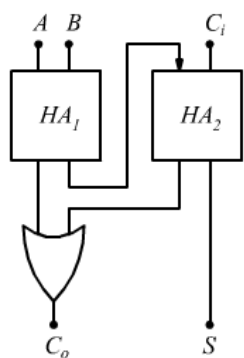


$$S = A \oplus B$$

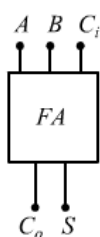
$$C = A \cdot B$$

A e B sono i bit da sommare, S è il bit della somma (sum) e C è il bit del riporto (carry).

Full-Adder



ingressi			uscite	
A	B	C _i	S	C _o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



Per ottenere la somma completa tra due numeri di più cifre, oltre ai bit dello stesso ordine occorre sommare anche il riporto eventualmente ottenuto dai due bit di ordine immediatamente inferiore, per questo motivo il circuito full-adder si presenta con tre ingressi e due uscite.

$$S = (A \oplus B) \oplus C_i$$

$$C_o = C_i(A \oplus B) + AB$$

I due ingressi sono costituiti dai due bit A_n e B_n da sommarsi e dal riporto C_i eventualmente ottenuto dalla somma dei due bit A_{n-1} e B_{n-1} .

Le due uscite sono composte dal bit di somma dei tre ingressi e dall'eventuale riporto C_o da inviare al full-adder successivo.

Qual è l'utilità e la convenienza dello stack pointer?

Lo stack pointer (SP) contiene l'indirizzo dell'ultima cella di memoria occupata nello stack. È conveniente perché grazie ad esso è possibile identificare la cima dello stack ed accedere facilmente alle parole di memoria utilizzando un offset.

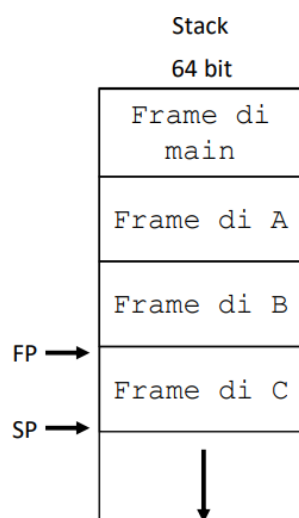
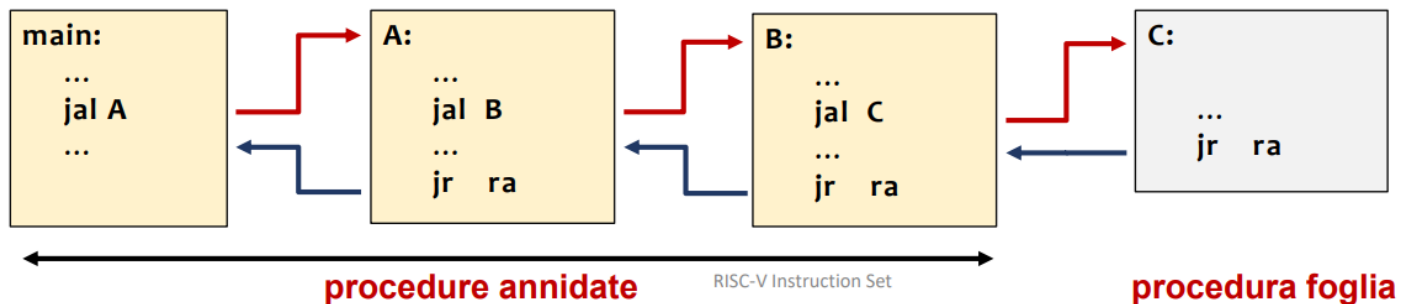
Spiegare anche FP, GP, PC

- **Frame Pointer (FP):** serve ad identificare la base di un frame
- **Global Pointer (GP):** serve ad identificare l'indirizzo della parte della memoria dedicata ai dati statici (globali)
- **Program Counter (PC):** serve ad identificare l'indirizzo della prossima istruzione da eseguire

Spiegare come funziona il record di attivazione

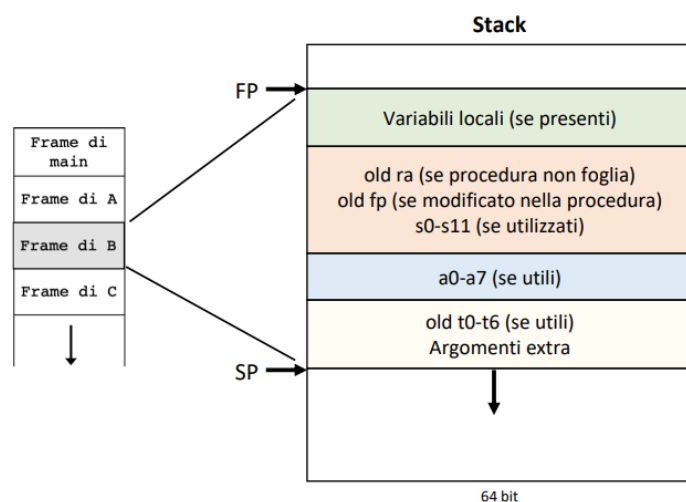
Ad ogni esecuzione di procedura corrisponde un blocco di memoria all'interno dello stack che viene chiamato **record di attivazione o frame**.

Quando viene richiamata una funzione F, viene aggiunto un record di attivazione allo stack. Quando la funzione F termina, il corrispondente record di attivazione viene cancellato.



Il frame in cima allo stack rappresenta il frame della procedura in esecuzione. I confini del frame sono identificati da:

- Base: il Frame Pointer (FP)
- Cima: lo Stack Pointer (SP)



Se utilizzato, il Frame Pointer viene inizializzato al valore di **sp** all'inizio della chiamata.

Fp consente di avere un riferimento alle variabili locali che non muta con l'esecuzione della procedura.

Se lo stack non contiene variabili locali alla procedura, il compilatore risparmia tempo di esecuzione evitando di impostare e ripristinare il frame.

Spiegare le varie fasi del passaggio dal codice C al linguaggio macchina

Programma C (livello 4)

-> **Compilatore** -> Programma in linguaggio assembler (livello 3)

-> **Assembler** -> Oggetto: modulo in linguaggio macchina + Oggetto: funzioni di libreria (livello 1)

-> **Linker** -> Eseguibile: programma in linguaggio macchina

-> **Loader** -> Memoria

Linguaggio Assemblativo: linguaggio le cui istruzioni sono ottenute dalle istruzioni ISA sostituendo i codici binari con **codici mnemonici**; il linguaggio assemblativo è quindi molto vicino al linguaggio macchina, c'è sostanzialmente una corrispondenza uno-uno tra le istruzioni ISA e le istruzioni del linguaggio assemblativo.

Per passare dal programma scritto in linguaggio **assemblativo** al programma **eseguibile** in linguaggio macchina si utilizza un programma traduttore detto assembler (**assembler**) che traduce i codici mnemonici nei codici numerici corrispondenti alle istruzioni ISA.

L'assembler legge tutte le istruzioni del programma assemblativo, ne traduce in linguaggio macchina i codici operativi, i dati e le label, controllandone la correttezza sintattica e restituisce in output il file **oggetto**.

Livello implementato tramite **compilazione** e NON interpretazione.

Assembler a due passi

- Passo 1:
 - Individuazione di tutti i **nomi** (le etichette) che compaiono come riferimento simbolico di dati o di istruzioni.
 - Creazione di una **Symbol Table** che contiene le etichette con la loro posizione relativa all'interno del programma
- Passo 2:
 - Traduzione dei codici mnemonici delle istruzioni, degli operandi e delle etichette, mediante la consultazione della **Symbol Table** costruita nel passo 1.

Linking

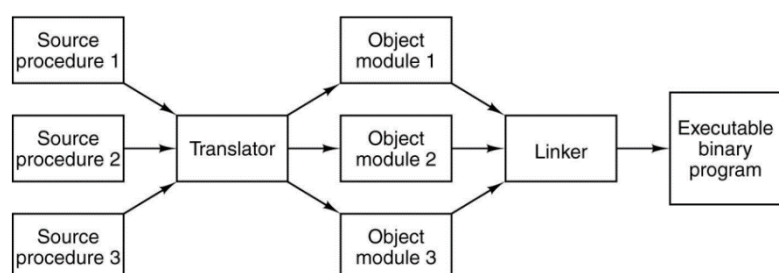
Programmi: insieme di procedure (**moduli**) tradotti separatamente dall'assembler (o compilatore). Ogni modulo oggetto ha il suo **spazio di indirizzamento** separato.

Linker: è un programma che esegue la funzione di **collegamento** dei moduli oggetto in modo da formare un unico **modulo eseguibile**.

Il linker **fonde** gli spazi di indirizzamento separati dei moduli oggetto in uno **spazio lineare unico** nel modo seguente:

- Costruisce una tabella di tutti i moduli oggetto e le loro lunghezze
- Assegna un indirizzo di inizio ad ogni modulo oggetto
- Trova tutte le istruzioni che accedono alla memoria e aggiunge a ciascun indirizzo una **relocation constant** corrispondente all'indirizzo di partenza del suo modulo
- Trova tutte le istruzioni che fanno riferimento ad altri moduli e le aggiorna con l'indirizzo corretto

Modulo	Lunghezza	Indirizzo partenza
A	400	100
B	600	500
C	500	1100
D	300	1600



Loading

Una volta creato l'eseguibile (ad opera del linker) esso viene memorizzato su un supporto di memoria secondaria. Al momento dell'esecuzione il sistema operativo lo carica in memoria e ne avvia l'esecuzione.

Il **loader** si occupa di:

1. Leggere l'intestazione per determinare la dimensione del programma e dei dati
2. Riservare uno spazio in memoria sufficiente per contenerli
3. Copiare programma e dati nello spazio riservato
4. Copiare nello stack i parametri (se presenti) passati al main
5. Inizializzare tutti i registri e lo stack pointer (ma anche gli altri del modello di memoria)
6. Saltare ad una procedura che copia i parametri dallo stack ai registri e che poi invoca il main

Quali sono le differenze tra linking statico e dinamico? Quali sono i vantaggi e svantaggi di ognuno?

Se si spostano in memoria programmi per cui è già stato fatto il collegamento e il calcolo degli indirizzi, tutti gli indirizzi di memoria risultano sbagliati e le informazioni di rilocalizzazioni sono state scartate da tempo. Il problema di spostare in memoria programmi è connesso con la scelta del momento in cui effettuare il collegamento (**binding**) tra nomi simbolici e indirizzi fisici.

Linking statico

Il linking statico si riferisce alla copia di tutte le librerie necessarie al programma **direttamente nel file eseguibile** tramite un linker. Ha luogo alla fine della fase di **compilazione**.

Vantaggi del linking statico

- **Tempi di esecuzione più rapidi:** poiché tutti i moduli vengono compilati in un unico file eseguibile, i programmi possono essere eseguiti più rapidamente
- **Migliore gestione della memoria:** il collegamento statico consente una migliore gestione della memoria, poiché tutti i moduli sono collegati direttamente tra loro
- **Utilizzo risorse ridotto:** il linking statico aiuta a ridurre la quantità di risorse necessarie per l'esecuzione dei programmi, poiché tutte le librerie sono già collegate tra loro

Svantaggi del linking statico

- **Difficile da mantenere:** se viene rilasciata una nuova versione, un programma che carica staticamente le librerie continua a utilizzare la vecchia versione
- **Dimensione dei file:** le librerie possono essere molto più grandi del programma; i file binari diventano eccessivamente grandi

Linking dinamico

Nel linking dinamico il sistema operativo carica i file necessari (librerie condivise) in memoria solo quando il programma è in esecuzione.

Vantaggi del linking dinamico

- **Tempi di caricamento più rapidi:** i componenti vengono caricati solo quando sono necessari
- **Scalabilità migliorata**
- **Manutenzione più semplice**
- **Requisiti di memoria ridotti**

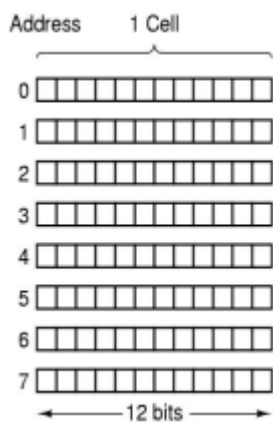
Svantaggi del linking dinamico

- **Fragilità:** a causa della natura modulare del collegamento dinamico, qualsiasi modifica al programma principale può portare al mancato caricamento dei componenti o all'utilizzo di librerie incompatibili
- **Problemi di dipendenza:** quando i collegamenti si interrompono a causa di dipendenze mancanti o non aggiornate, possono verificarsi problemi di esecuzione.
- **Prestazioni ridotte:** alcuni processi possono richiedere più tempo se le librerie a collegamenti dinamici sono state richiamate più di una volta durante l'esecuzione
- **Versioni inaffidabili:** utilizzare librerie di terze parti significa non avere alcun controllo sulla loro stabilità e accuratezza, quindi qualsiasi modifica potrebbe influire sulla correttezza del risultato del codice

Cosa significa invocare il main?

Il main generalmente è la prima funzione eseguita durante l'avvio di un programma. Invocare il main significa avviare l'esecuzione di esso, a seguito dell'operazione di **loading** del programma in memoria da parte del sistema operativo.

Come si recupera qualcosa dalla memoria?



La memoria principale è un insieme di celle (o locazioni), tutte le celle hanno la stessa dimensione ed ogni cella è identificata da un indirizzo, la cella è la minima unità indirizzabile.

Se un calcolatore ha **n** celle allora gli indirizzi vanno da **0** a **n-1 indipendentemente** dalla dimensione della cella.

Il numero di bit nell'indirizzo determina il numero massimo di celle indirizzabili.

Parola: sequenza di bit, unità di manipolazione per le istruzioni

La CPU è connessa alla memoria tramite due bus:

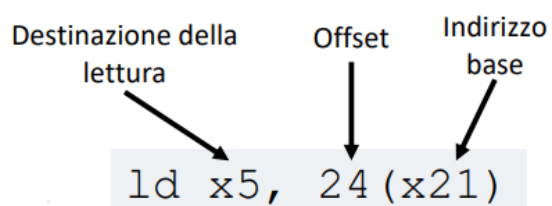
- **Indirizzo:** **n** piedini corrispondono a 2^n locazioni di memoria indirizzabili (esempio: 32, 64)
- **Dati:** **n** piedini permettono di leggere/scrivere una parola di **n** bit con una sola operazione (esempio: 32, 64)

In lettura la CPU:

- Attiva la linea del bus di controllo che corrisponde alla lettura
- Trasmette sul bus indirizzo l'indirizzo della cella cui vuole accedere
- Con un piccolo ritardo (tempo di accesso) riceve sul bus dati il contenuto della cella

Ad esempio in RISC-V, l'istruzione load copia un dato dalla memoria ad un registro. L'indirizzo del dato in memoria viene specificato da:

- Indirizzo base (contenuto in un registro)
- Scostamento o offset (compreso tra -2048 e +2047)



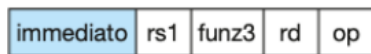
L'istruzione **ld** (load doubleword) carica una parola doppia (64 bit) dalla memoria in un registro.

Quali sono i modi di indirizzamento?

I modi di indirizzamento sono convenzioni utilizzate nella programmazione in linguaggio assembly che specificano come certi valori vengono letti in un determinato segmento di istruzioni per il montaggio. Abbiamo quattro modi di indirizzamento principali:

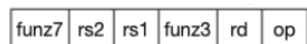
- Indirizzamento immediato
- Indirizzamento tramite registro
- Indirizzamento tramite base e spiazzamento
- Indirizzamento relativo al program counter

Indirizzamento immediato



L'indirizzamento immediato si ha quando all'interno di un'istruzione si opera con una costante numerica.

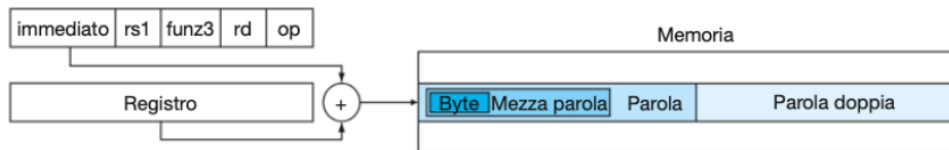
Indirizzamento tramite registro



L'indirizzamento a registro si ha quando gli operandi dell'istruzione sono entrambi registri della CPU.

Indirizzamento tramite base e spiazzamento

L'indirizzamento tramite base e spiazzamento è utilizzato per recuperare un dato dalla memoria che si trova ad un offset (campo immediato) rispetto ad un indirizzo contenuto nel registro utilizzato nell'istruzione.



Indirizzamento relativo al program counter

L'indirizzamento relativo al program counter viene utilizzato per ottenere il prossimo valore di PC in caso di salto, eseguendo il calcolo **PC = PC + immediato**.

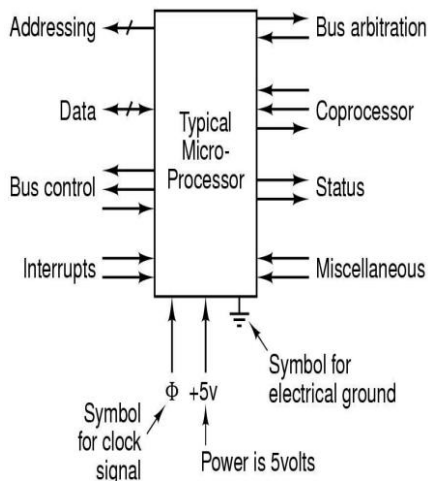


Quali sono i tipi di bus?

Il bus è l'insieme di linee elettriche che collegano i moduli di un elaboratore. Le linee del bus possono essere linee di dati, di indirizzo e di controllo.

- **Linee di dati:** è detto data bus. Il numero di linee (**larghezza** del data bus) determina il numero di bit che possono essere trasmessi alla volta: ha un impatto sulle prestazioni del sistema.
- **Linee di indirizzo:** permettono di individuare la sorgente/destinazione dei dati trasmessi sul data bus.
- **Linee di controllo:** controllano l'accesso e l'utilizzo delle linee di dati e di indirizzo

Connessioni di una CPU



- **Indirizzo:** n piedini corrispondono a 2^n locazioni di memoria indirizzabili
- **Dati:** n piedini permettono di leggere/scrivere una parola di n bit con una sola operazione
- **Controllo:** regolano il flusso e la scansione dei dati verso e dal chip, categorie principali:
 - Controllo del bus
 - Interrupt
 - Arbitraggio del bus
 - Segnali del coprocessore
 - Stato
 - Varie

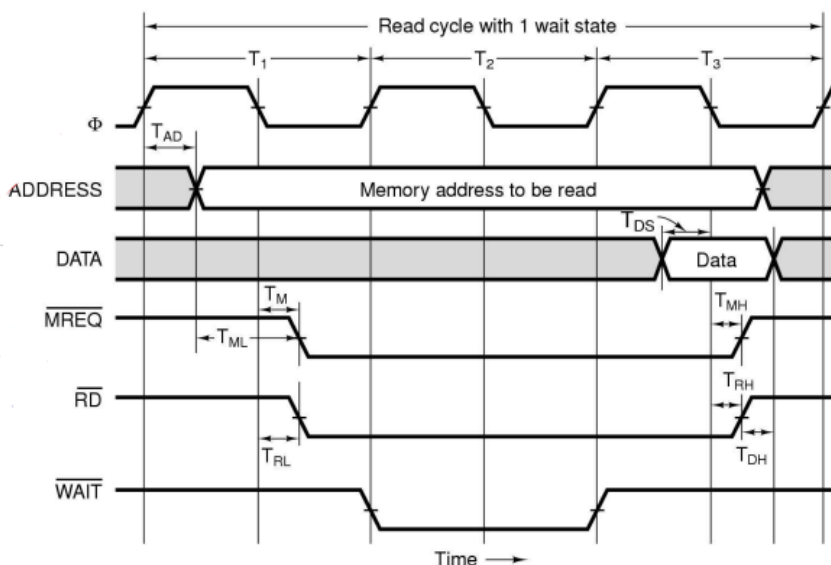
Bus “larghi” sono più costosi di quelli stretti ma offrono una banda più larga e quindi maggiore velocità di trasferimento. Maggiore velocità si ottiene aumentando la larghezza del bus (più bit/trasferimento) o diminuendo il ciclo di bus (più trasferimenti/secondo).

Per ovviare ai problemi dati da bus molto larghi talvolta si opta per un **multiplexed bus**: le linee utilizzate per il trasferimento dei dati e degli indirizzi sono le stesse; prima si trasmettono gli indirizzi e poi i dati. Ovviamente il bus multiplexato è più lento.

I bus si possono dividere in due categorie ben distinte:

- **Bus sincroni:** hanno una linea pilotata da un oscillatore, che ha una determinata frequenza. Tutte le attività del bus richiedono un numero intero di questi cicli.
- **Bus asincroni:** non hanno un clock principale; i cicli del bus possono essere della lunghezza necessaria e non devono essere uguali per tutti i dispositivi.

Bus sincrono – ciclo di lettura



La durata delle fasi è nota ad entrambi i partecipanti e l'unica incognita è l'inizio della comunicazione.

- La linea **MREQ** indica l'inizio della comunicazione
- La CPU mette l'indirizzo sul bus dopo un certo tempo necessario per stabilizzarsi (T_{AD}), l'indirizzo rimane disponibile
- A linee di indirizzo stabili la CPU attiva i segnali di richiesta di **inizio comunicazione** (linea MREQ) e il **segnale di lettura** (linea RD)

- Per avvertire la CPU di non aspettare i dati, all'inizio del ciclo T_2 , la **memoria** attiva il segnale di **attesa dati** (linea WAIT)
- La memoria mette i dati sulla linea dati
- Dopo aver letto i dati la CPU disattiva i segnali di **richiesta** (linea MREQ) e di **lettura** (linea RD)

Vantaggi del bus sincrono

- Realizzazione device semplice
- Se la durata di un'operazione è fissa non occorre una linea di wait

Svantaggi del bus sincrono

- Durata di un'operazione deve necessariamente avere una durata pari ad un numero intero di cicli
- Il bus sincrono deve adattarsi al dispositivo più lento

Bus asincrono

- Attivati i segnali di accesso alla memoria e lettura, attiva il segnale SYNchronization (linea MSYN)
- Il device esegue il suo lavoro all'attivazione del segnale e quindi terminato attiva il segnale di SYNchronization (linea SSYN)
- I dati vengono memorizzati dal device che nega i segnali di **accesso alla memoria, lettura e SYNchronization**

Full handshake: Il device dopo aver visto la negazione del segnale di synchronization, nega a sua volta il suo segnale di SYNchronization

Vantaggi del bus asincrono

- Flessibilità: la durata di un'operazione è determinata unicamente dalla velocità della coppia di device

Svantaggi del bus asincrono

- Per completare un'operazione di comunicazione sono sempre necessarie 4 azioni
- Occorre inserire nei device i circuiti necessari a rispondere opportunamente al protocollo

Come funziona il daisy chaining?

Il daisy chaining è un tipo di arbitraggio centralizzato dei bus.

L'arbitraggio dei bus è la funzione che gestisce il possesso del bus per evitare ambiguità quando più master richiedono contemporaneamente il suo utilizzo; l'arbitro decide a quale dispositivo concederlo per prima.

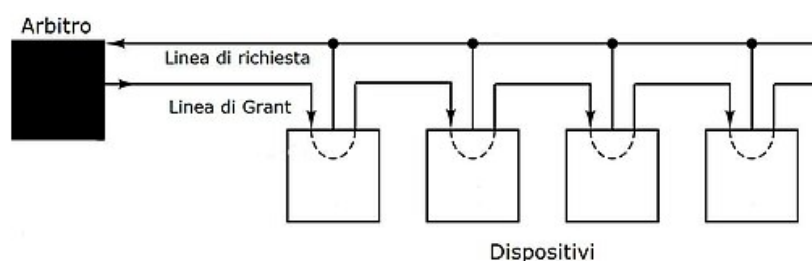
Esistono due tipi di meccanismi di arbitraggio: **centralizzato o decentralizzato**.

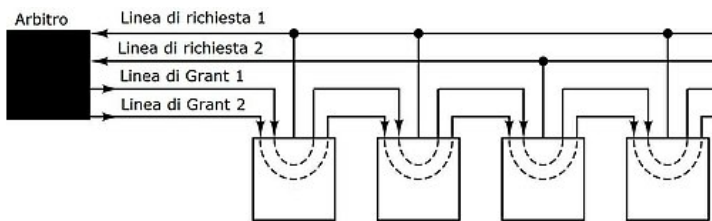
Nell'arbitraggio centralizzato il bus viene concesso da un circuito integrato nella CPU chiamato appunto arbitro, invece nell'arbitraggio decentralizzato non esiste questo chip, la concessione del bus viene gestita dagli stessi dispositivi.

Nel daisy chaining, quando un dispositivo riceve il **segnale di Grant** (segnale di concessione del bus), controlla se ha fatto una richiesta del bus:

- Se ha fatto la richiesta, utilizza il bus e non propaga il segnale di Grant ai dispositivi successivi
- Se non ha fatto la richiesta, propaga il segnale di Grant

Quindi in un meccanismo Daisy Chaining il dispositivo fisicamente più vicino all'arbitro ha più priorità rispetto ai dispositivi più lontani.

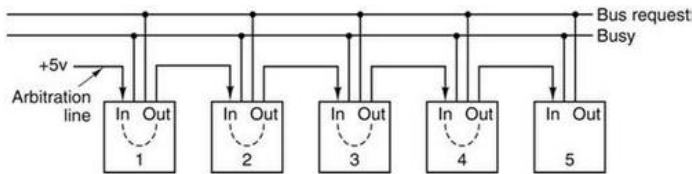




È anche possibile avere più livelli di priorità, caratterizzati ognuno da una propria linea di Richiesta del bus e una linea di Grant.

Nell'arbitraggio decentralizzato invece:

- Non esiste l'arbitro
- Ogni unità funzionale ha priorità fissata e diversa, espressa da un codice univoco associato all'unità stessa
- Se più dispositivi hanno bisogno del bus, questo viene preso da chi ha priorità maggiore



Questo tipo di arbitraggio utilizza solo 3 linee:

- Linea di richiesta, che effettua la richiesta del bus
- Busy, è attivata dal master che ne fa richiesta
- Linea di arbitraggio, che collega tutti i dispositivi

Quando nessuno richiede il bus, la linea di arbitraggio si propaga per tutti i dispositivi, mentre se un dispositivo lo richiede, dovrà controllare se il bus è libero e se il segnale della linea di arbitraggio (IN) è attivato; in questo caso nega OUT in modo che tutti i dispositivi successivi vedranno IN negato e a loro volta negheranno OUT. In questo modo il dispositivo con priorità maggiore che ha fatto richiesta, avrà il possesso del bus, attiverà la linea di Busy e OUT, e inizierà il proprio trasferimento. Questo schema è simile a quello Daisy Chaining a differenza che è privo di arbitro, questa differenza lo rende più economico, veloce e non soggetto a eventuali guasti da parte dell'arbitro.

Cos'è e come funziona il complemento a due? Quale problema risolve e perché è il più comodo?

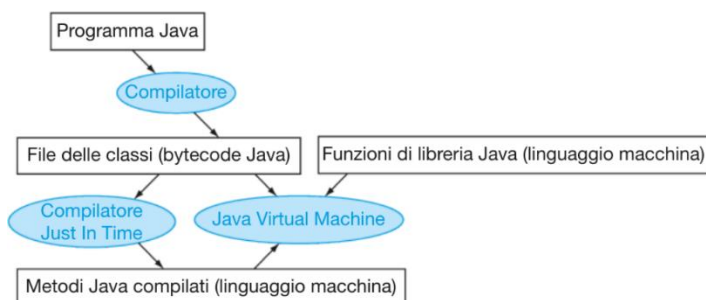
Il complemento a due è un metodo per la rappresentazione dei numeri binari con segno. Nel complemento a due, il bit più significativo del numero ha peso negativo o positivo; da questo deriva che tutti i numeri che cominciano con un "1" sono numeri binari negativi, mentre tutti i numeri che cominciano con uno "0" sono numeri binari positivi.

Un numero binario di n cifre può rappresentare in questo modo i numeri compresi fra $-2^{(n-1)}$ e $+2^{(n-1)} - 1$.

Ad esempio, un numero binario di 8 bit può rappresentare i numeri compresi tra -128 e +127

Il vantaggio principale del complemento a due è dato dal fatto che i circuiti di addizione e sottrazione non devono esaminare il segno di un numero rappresentato con questo sistema per determinare quale delle due operazioni sia necessaria, permettendo tecnologie più semplici e con maggiore precisione; si utilizza un solo circuito, il sommatore, sia per l'addizione che per la sottrazione.

Come funziona java (i vari passaggi per arrivare al linguaggio macchina)?



I passaggi per arrivare al linguaggio macchina sono:

- Compilazione del codice, che restituisce come risultato il cosiddetto **bytecode** dell'applicazione
- Interpretazione del bytecode da parte della JVM

La JVM può invocare il compilatore Just In Time (JIT), che compila i metodi del linguaggio Java nel linguaggio macchina del calcolatore sul quale è in esecuzione.

Quali sono i parametri per stabilire se un computer è più veloce?

Tempo di risposta: detto anche **tempo di esecuzione**, è il tempo totale richiesto da un calcolatore per completare un task; esso comprende gli accessi a disco, gli accessi a memoria, le attività di I/O, il tempo richiesto dal sistema operativo, il tempo d'esecuzione della CPU, ecc.

Throughput: detto anche **bandwidth** (larghezza di banda), è un'altra misura delle prestazioni e rappresenta il numero di programmi completati per unità di tempo.

Un computer è più veloce rispetto ad un altro nell'esecuzione di un task se il suo tempo di esecuzione per quel task è inferiore.

$Prestazioni_x > Prestazioni_y$ se $Tempo\ di\ esecuzione_x < Tempo\ di\ esecuzione_y$

Tempo di esecuzione della CPU: detto anche **tempo di CPU**, è il tempo che la CPU utilizza effettivamente nella computazione richiesta da un certo task.

Periodo di clock: durata di un ciclo di clock

$$\frac{\text{Tempo di CPU relativo a un programma}}{\text{Cicli di clock della CPU relativi al programma}} = \text{Periodo di clock}$$

Ci sono istruzioni più veloci di altre?

Sì, ci sono istruzioni che sono più veloci di altre, questo dipende dal numero di **cicli di clock per istruzione (CPI)**, ovvero il numero medio di cicli di clock per istruzione di un programma o di un frammento di programma.

Perché il Bus è fonte di ritardo nell'esecuzione dell'istruzione?

Il bus è fonte di ritardo nell'esecuzione dell'istruzione a causa di diversi fattori:

- Propagazione dei segnali elettrici
- Tempo di stabilizzazione dei segnali
- Tempo di accesso alla memoria
- Tempi di attesa

Quali e cosa sono le Trap?

Le trap (o eccezioni o interrupt sincroni) sono chiamate a procedure automatiche che possono essere causate dal verificarsi di qualche **condizione eccezionale** oppure da istruzioni apposite per richiedere **servizi di base del sistema operativo**. Le trap sono **sincrone** al programma.

Gestore delle trap: procedura di gestione della richiesta di trap il cui indirizzo è memorizzato in una locazione fissa a cui il programma salta in caso di trap.

Tipi di eccezioni (eventi sincroni):

- **Errore:** eccezione causata da eventi interni (overflow, divisione per zero, ecc.)
- **Environment Call:** causata da richiesta di un Servizio di Sistema (stampa di un messaggio, lettura di un intero, ecc.)
- **Environment break:** causata da motivi diagnostici o debugging

Cosa fa la CPU quando c'è una Trap?

Quando si verifica una trap, la CPU salva il punto del codice (Program Counter) in cui si è verificata l'eccezione, il controllo passa quindi all'Exception Handler, che deve rimediare alla situazione eseguendo azioni diverse a seconda della causa dell'eccezione identificata.

Exception Handler: Il gestore dell'eccezione deve essere eseguito in **modalità protetta (kernel mode)**, deve evitare di modificare lo stato dell'applicazione (i contenuti di **tutti** i registri non devono essere alterati) e gestisce le eccezioni una alla volta.

Come sono fatti i diversi tipi di memoria? Anche nello specifico i diversi tipi di RAM

Ci sono due tipi principali di RAM:

- RAM Statica (SRAM)
- RAM Dinamica (DRAM)

RAM Statica (SRAM)

Le SRAM sono semplicemente dei circuiti integrati organizzati come vettori di memoria che (di solito) hanno una sola porta di accesso che può fornire sia la lettura sia la scrittura.

Vantaggi delle SRAM:

- Le SRAM hanno uno stesso tempo di accesso per tutti i dati, anche se i tempi di accesso in lettura e scrittura possono essere diversi.
- Le SRAM non hanno bisogno di essere rinfrescate, per cui il loro tempo di accesso è molto vicino al periodo del clock;
- Le SRAM hanno bisogno di una potenza minima per mantenere la carica quando si trovano in modalità stand-by

Svantaggi delle SRAM:

- Utilizzano da sei a otto transistor per bit per evitare che l'informazione possa essere disturbata quando viene letta, diminuendo la densità della memoria ed aumentando il costo per bit

RAM Dinamica (DRAM)

In una memoria RAM dinamica, il dato viene memorizzato come carica in un condensatore e un solo transistor è sufficiente per leggere il dato o per sovrascriverlo.

Vantaggio delle DRAM: Dato che le DRAM utilizzano un solo transistor per bit memorizzato, sono molto più dense e il costo per bit è inferiore a quello delle SRAM

Svantaggio delle DRAM: Dato che nelle DRAM l'informazione viene memorizzata in un condensatore, non rimane indefinitamente e occorre rinfrescarla periodicamente

Per migliorare ulteriormente l'interfaccia con i processori, alle DRAM è stato aggiunto il clock (Synchronous DRAM). Il vantaggio delle SDRAM sta nel fatto che il clock elimina il tempo necessario a sincronizzare la memoria con il processore. L'aumento di velocità delle DRAM sincrone proviene dalla possibilità di trasferire gruppi di bit adiacenti a raffica senza dovere specificare altri indirizzi. Le SDRAM sono strutturate a **banchi**.

Altri tipi di memoria: memorie flash, memorie a disco.

Le **memorie flash** sono un tipo di memoria programmabile a sola lettura, cancellabile elettricamente. A differenza dei dischi e delle DRAM, i bit delle memorie flash si deteriorano dopo un certo numero di scritture.

Le **memorie a disco** sono formate da un gruppo di dischi, detti piatti, che ruotano ad una predeterminata velocità. Ciascun piatto metallico è ricoperto su entrambe le facce da materiale magnetico registrabile. Per leggere e scrivere dati su un disco, al di sopra di ognuna delle due superfici di ogni disco è posizionato un braccio mobile contenente una piccola bobina elettromagnetica chiamata **testina di lettura/scrittura**.

La superficie di ogni disco è divisa in cerchi concentrici chiamati **tracce**, ogni traccia è a sua volta suddivisa in **settori**, che sono l'unità di memorizzazione delle informazioni.

Per accedere ai dati, il sistema operativo effettua un'operazione chiamata **seek**, e il tempo necessario per spostare la testina sopra la traccia desiderata viene chiamato tempo di ricerca (**seek time**).

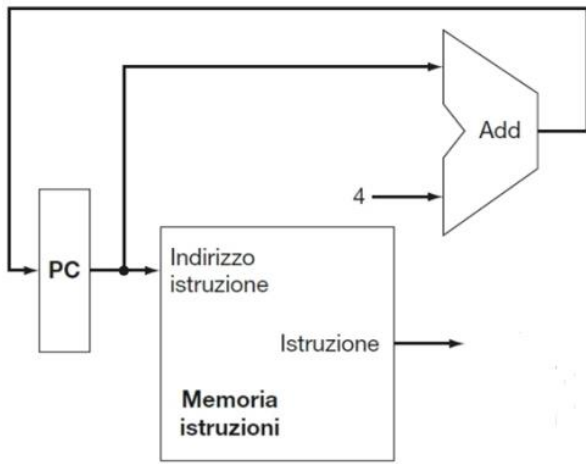
Quando la testina di lettura/scrittura ha raggiunto la traccia corretta, deve aspettare che il settore desiderato passi sotto. Questo tempo di attesa viene chiamato **latenza di rotazione** o **ritardo di rotazione**.

Cosa sono le convenzioni di chiamata?

Le convenzioni di chiamata sono degli schemi a livello di implementazione per il modo in cui le funzioni ricevono i parametri dal loro chiamante e come restituiscono un risultato. Le convenzioni di chiamata ad esempio possono specificare:

- In quali registri sono posizionati parametri, valori di ritorno e indirizzi di ritorno
- Come viene restituito un valore di ritorno dal chiamato al chiamante (nello stack, in un registro, ecc.)

Come viene aggiornato il Program Counter?



Nella maggior parte dei casi, il Program Counter viene aggiornato grazie ad un sommatore, che aggiunge 4 byte all'indirizzo corrente, per ottenere l'indirizzo dell'istruzione successiva (questo perché tutte le istruzioni RISC-V sono rappresentate con 32 bit).

In caso di salto, invece, il Program Counter assume il valore dell'indirizzo di destinazione specificato dall'istruzione di salto.

Qual è la differenza tra Flip Flop e Latch?

1. Il fattore chiave di differenziazione tra il latch e il flip flop è che il latch cambia continuamente l'uscita in base alla variazione dell'ingresso, mentre un flip flop cambia l'uscita solo quando viene attivato l'impulso di clock insieme alla modifica dell'ingresso
2. In latch poiché l'uscita dipende solo dall'ingresso applicato, non è necessario un clock esterno. Invece, i flip flop richiedono necessariamente un segnale di clock esterno per il loro funzionamento
3. L'assenza di un clock fa sì che i latch funzionino in modo asincrono mentre, poiché il segnale di clock sincronizza il funzionamento dei flip flop, il flip flop funziona in modo sincrono
4. I latch sono **attivati a livello**, quindi funzionano ogni volta che l'ingresso cambia da un livello binario all'altro. I flip flop sono **attivati sul fronte**, quindi si attivano quando il segnale di clock passa da basso ad alto o da alto a basso.

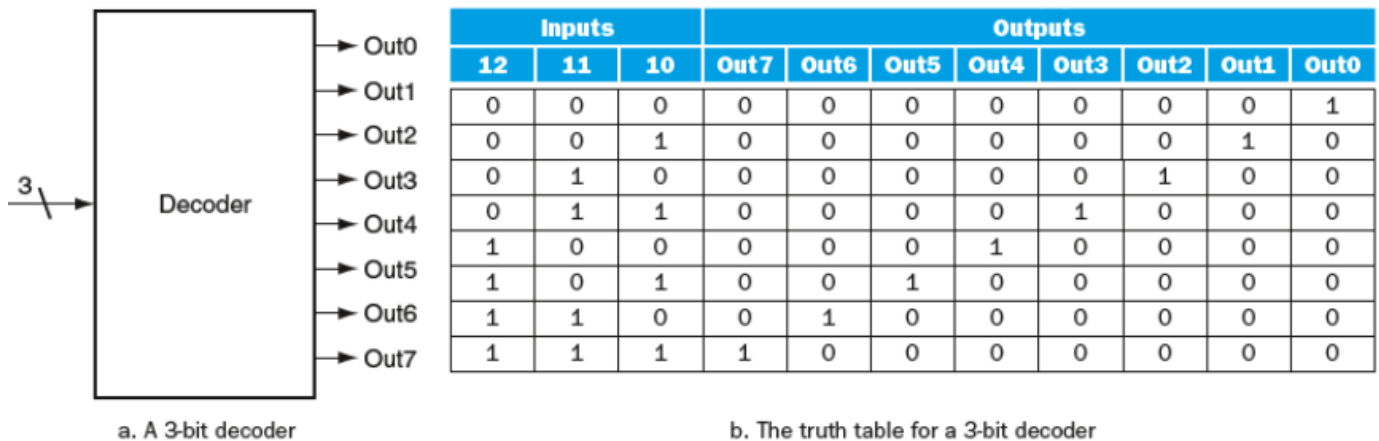
Cos'è una macchina a stati finiti? Ha circuiti sequenziali o combinatori? Da cosa dipende lo stato futuro di questa macchina?

Una macchina a stati finiti è un dispositivo automatico in grado di interagire con l'ambiente esterno. A fronte di un input, esibisce un output che dipende anche da **informazioni memorizzate in elementi interni** (stati). Quindi possiede al suo interno circuiti sequenziali. Lo stato futuro di una macchina a stati finiti dipende dall'input e dallo stato attuale.

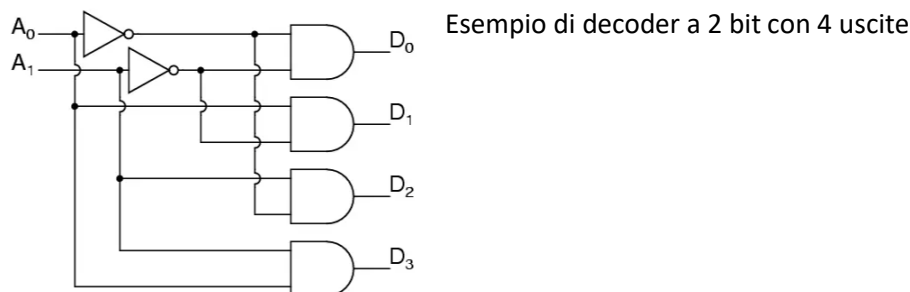
s: $AxQ \rightarrow Q$ funzione di cambiamento di stato A = simbolo in ingresso, Q = simbolo di stato

Cos'è il decoder e come funziona?

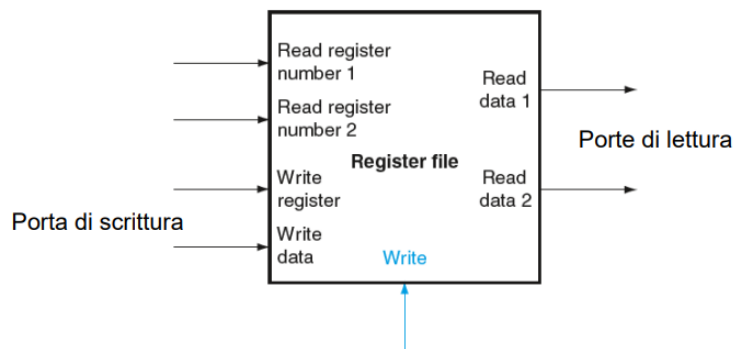
Il decoder è un circuito combinatorio che prende un numero di n bit come ingresso e lo usa per selezionare (mettere a 1, asserire) una delle 2^n linee di uscita. Può essere utilizzato per attivare una certa componente, un banco di memoria, ecc.



Gli ingressi vanno interpretati come le cifre di un numero in base 2 con 12 che è la cifra più significativa.



Come funziona il blocco dei registri (Register File)?

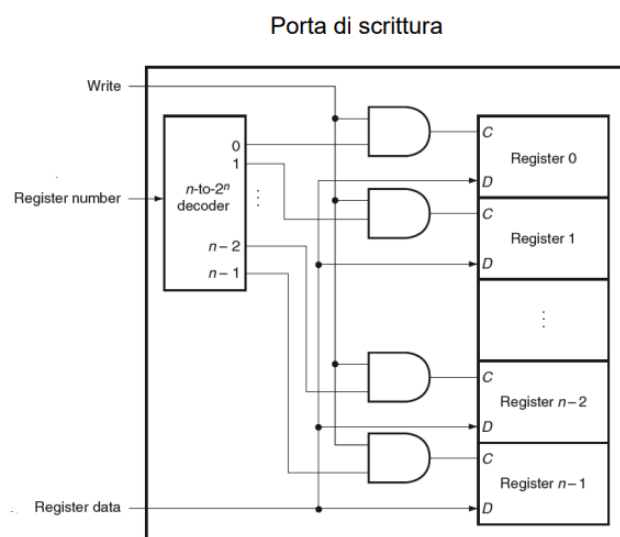


Il blocco di registri è un insieme di registri leggibili e scrivibili.

Si deve fornire in ingresso il numero del registro sul quale compiere l'operazione.

Si implementa con un multiplexer per ogni porta di lettura, con un decoder per la porta di scrittura ed un insieme di registri basati su flip-flop di tipo D.

Come collabora il decoder con il blocco dei registri?



Nel blocco dei registri, il decoder viene utilizzato per stabilire su quale registro scrivere quando il segnale di Write è abilitato.

Ha come input il numero del registro in binario e l'uscita di output è collegata al registro corrispondente.

Cos'è l'overflow?

L'overflow è un problema relativo alle operazioni eseguite sui numeri da un calcolatore. Il problema è dovuto all'impossibilità dei calcolatori di memorizzare numeri di dimensione arbitrariamente grande. Esso si verifica quando il risultato di un'operazione eccede il valore massimo rappresentabile con n bit fissati a disposizione in memoria.

In complemento a 2 si ha overflow quando:

- Ci sono addendi dello stesso segno e il segno del risultato è diverso dal segno degli addendi
- Il riporto dalla colonna n-2 alla colonna n-1 a quella oltre la cifra più significativa sono discordi (uno dei due è 0 e l'altro è 1)

Esistono due modi diversi di gestione dell'overflow:

- **Senza trap:** l'hardware setta un bit in un apposito registro e il programmatore ISA, se lo desidera, dopo un'istruzione che può causare overflow, testa il registro e salta ad apposita procedura
- **Con trap:** la condizione di overflow (rilevata dall'hardware) genera una trap, che modifica il flusso di controllo ad una locazione prefissata (gestore della trap)

Cosa c'è dentro una ALU?

All'interno di una ALU ad 1 bit abbiamo:

- 2 ingressi dati da 1 bit, A e B, i bit su cui vogliamo eseguire l'operazione
- 1 ingresso di controllo da 2 bit **Operation**, per selezionare l'operazione desiderata
- 1 ingresso da 1 bit **CarryIn**, per il riporto della somma
- 1 uscita da 1 bit per il risultato dell'operazione eseguita
- Multiplexer per selezionare il risultato dell'operazione desiderata
- 1 porta AND
- 1 porta OR
- 1 circuito Half-Adder

